

~~BLUE~~ WEB APPLICATIONS : BASICS

- HTTP messages are of two type : a) HTTP request : Client to web server
(b) HTTP response : Webserver to client
- What forms an HTTP message?
 - a) Start line : Tells whether its a request or response message
 - b) Header : they cover all things like security, content types, and more
 - c) Empty line
 - (d) Body : The actual data of the message

- HTTP request :

GET /user/login.html HTTP/1.1

The diagram shows the request line 'GET /user/login.html HTTP/1.1' with three brackets underneath. The first bracket is under 'GET' with an arrow pointing to 'HTTP method'. The second bracket is under '/user/login.html' with an arrow pointing to 'Path'. The third bracket is under 'HTTP/1.1' with an arrow pointing to 'HTTP version'.

- HTTP methods :

- a) GET
- (b) PUT
- (c) POST
- (d) DELETE
- e) PATCH : updates a small part of a resource

• What is injection?

It is a type of cyberattack where the attacker inserts malicious code into a web application, API etc to be executed

Page No.	
Date:	

the interpreter.

(f) HEAD: works like GET but only retrieves headers, not the full content.

(g) OPTIONS: Tells you what methods are available for a specific resource.

(h) TRACE: Similar to OPTIONS, shows which methods are allowed, often for debugging.

i) CONNECT: used to create a secure connection, like for HTTPS.

• URL Path: It tells which resource ^{where to find the resource} from the webserver is the user trying to interact with.

• HTTP version: It shows the protocol version used to communicate between the client and the server. EX:

a) HTTP/0.9: The first version only supported GET request.

b) HTTP/1.0: Added headers and better support for content.

c) HTTP/1.1: Brought persistent connect, chunked transfer encoding.

d) HTTP/2: Introduced features like multiplexing, header compression.

e) HTTP/3: Build on HTTP/2 but uses new protocol.

• HTTP Request Headers:

Request headers convey extra information to the webserver about the request.

Example

a) HOST: HOST: tryhackme.com Specifies the name of the server the request is for.

(b) User-Agent: User-Agent: Mozilla/5.0 Shares information about the web browser the

request is coming from

(c) Referrer: Referrer: <https://www.google.com> Indicates the URL the request came from

(d) Cookie: Cookie: user_type=Student; Information the server previously asked the web browser to store is held in cookies
 xcom=intrataudrapp; xcom_status= in-progress

(e) Content-type: Content-type: Application/json Describes what type of data is in the ~~request~~ body

• Request body:

In Request like POST or PUT, the data sent to the web server is located inside request body. The formatting of data can take many ~~form~~ forms, like URL encoded, Form data, JSON or XML.

(a) URL encoded: A format where data is structured in pairs of keys and value where (key = value). Multiple pairs separated by (&). EX: name = Saromah & age = 18 & country = USA

(b) Form data: Allows multiple data blocks to be sent where each block is separated by a boundary string. It is used to send binary data, such as when uploading files and images to a web server

(c) JSON: Data is formatted in pairs of name: value. Multiple pairs are separated by commas, all contained within curly braces. EX: {

"name": "Saromah",
 "age": "17",
 }

(d) XML: In XML format data is ^{structured} ~~and~~ between tags which have opening and closing tags. Ex:

```
<user>
  <name>Saxonshe </name>
  <age> 17 </age>
</user>
```

• HTTP response:

The HTTP response includes a status code and a short explanation (called a reason phrase) that gives insight into how the server handled your request.

• Status line: The first line of HTTP response. It contains 3 key pieces of info: (a) HTTP version: Tells which version of HTTP is being used.

(b) Status code: A three digit no. showing outcome of your request.

(c) Reason phrase: A short message explaining the status code.

These codes fall into five categories:

- Information Response (100-199)
- Successful response (200-299)
- Redirection messages (300-399)
- Client error response (400-499)
- Server error response (500-599)

Common status codes: 100 (CONTINUE), 200 (OK), 301 (MOVED PERMANENTLY), 404 (NOT FOUND), 500 (INTERNAL SERVER ERROR)

* Make sure cookies are set with HttpOnly flag (so they can't be accessed by Javascript) and the Secure flag (so they are only sent over HTTPS)

Page No.	
Date:	

- **Response Headers:** These headers provides important information about HTTP response and tells the client (usually browser) how to handle it.

Some headers are crucial for making sure the HTTP response works properly.

Ex

(a) **DATE:** DATE: Fri, 23 Aug 2024 10:47:21 GMT

It shows the exact date and time when the server generated the response.

(b) **Content-type:** Content-type: text/html; charset=utf-8

It tells the client that what type of content its getting, it also include character set to help browser display it properly.

(c) **SERVER:** SERVER: nginx This header tells what kind of server software is handling the request.

(d) **SET-COOKIE:** SET-COOKIE: sessionid=33af... This sends the cookies from the server to the client which the client stores and send back with future request.

(e) **CACHE-CONTROL:** CACHE-control: max-age=600 It is used to control how responses are cached by your browser, proxies or caching servers.

(f) **Location:** Location: /index.html This one's tells the client where to go next if the resource has moved. Used when status code is: (300-399)

* XSS (Cross-site scripting): It is a web vulnerability where an attacker injects malicious JavaScript into a website and that script runs inside another user's browser when he requested the website.

Page No.	
Date:	

- **Response body:** The HTTP response body is where the actual data lives: things like HTML, JSON, images etc that the server sends back to the client.
 - **Security headers:** HTTP security headers help improve the overall security of the web application.
- Different types of security headers:

i) **Content-Security Policy (CSP):** It provides a way for administrators to say what domains or sources are considered safe and provides a layer of mitigation to such attacks.

Ex: Content-Security-Policy: default-src 'self'; script-src 'self'
 https://cdn.tryhackme.com; style-src 'self'

Self is a special keyword that reflects the same domain on which the website is hosted.

- **default-src:** which specifies the default policy of self, which means only the current website.
- **script-src:** which specifies the policy for where scripts can be loaded from, which is self along with scripts hosted on https://cdn.tryhackme.com
- **style-src:** which specifies the policy for where style CSS can be loaded from the current website.

ii) **Strict-Transport-Security (HSTS):**

The HSTS header ensures that web browsers ~~always connect~~ always connect over HTTP.

Ex: Strict-transport-security: max-age=63072000; includeSubdomains; preload

- max-age: This is the expiry time for this setting in seconds.
- includesubdomains: An optional setting that instructs the ~~web~~ browser to also apply this setting to subdomains.
- preload: It allows website to be included in preload lists. Browser can use preload list to enforce HTTPS before even having their first visit to a website.

iii) X-Content-Type-Options: It tells the browser not to guess MIME type of a resource but only use the Content-type header.

Example: X-Content-Type-Options: nosniff

- nosniff: Tells the browser not to sniff or guess the MIME type.

iv) Referrer-Policy: This header ~~also~~ controls the amount the amount of information sent to the destination web server when a user is redirected from the source webserver, such as when they click a hyperlink.

Ex: Referrer-Policy: no-referrer/sameorigin/strict origin/strict-origin-when-cross-origin

- no-referrer: This completely disables any info being sent about the referrer.

Page No.			
Date :			

- same-origin : This only sends referer info when the hyperlink is within the same website.
- strict-origin : It only sends the info when the protocol stays the same.
- strict-origin - when-cross-origin : This is similar to the strict-origin except for same-origin requests, where it sends the full URL path in the origin header.